Resilient Clinical Outcome Prediction from Noisy Medical Records: A Distributed PySpark Pipeline with Adversarial Noise Simulation

Nafeesa Mahek and Jumanah Al Nahdi and Aseel Bajaber and Hams Al Johani
Department of Computer Science, Effat University
Jeddah, Saudi Arabia
{nafeesa, jumanah, aseel, hams}@effatuniversity.edu.sa

Abstract

Predicting 30-day diabetic patient readmission is critical for healthcare intervention, but clinical datasets are notoriously noisy and prone to silent corruption. This paper presents a fully distributed PySpark pipeline that explicitly addresses adversarial data quality degradation. Using the UCI Diabetes 130-US Hospitals dataset (101,766 encounters), we simulate four realistic adversarial attacks: NULL injection, sentinel amplification, label flipping, and outlier injection. We demonstrate a robust seven-step cleaning protocol that successfully recovers 89.97% of the corrupted data. We then train three Spark MLlib classifiers, utilizing instance weighting to handle the severe class imbalance. Logistic Regression achieves the best F1 score of 0.71. Finally, scalability experiments confirm sub-linear training time growth, validating the pipeline's distributed efficiency and readiness for large-scale clinical deployment.

1 Introduction and Motivation

Hospital readmission within 30 days of discharge is one of the most costly and preventable events in modern healthcare. In the United States alone, hospitals pay over $26 billion annually in Medicare penalties tied to excess readmissions (Centers for Medicare and Medicaid Services, 2023). For diabetic patients specifically, the readmission rate is disproportionately high due to the chronic and multifactorial nature of the disease. Early prediction of which patients are at risk of readmission allows care teams to schedule follow-up appointments, adjust medications, and allocate resources more effectively to prevent patient deterioration.

At the same time, the data produced by clinical information systems is notoriously messy and inconsistent. Electronic health records (EHR) are often populated manually by nurses and physicians under severe time pressure, leading to missing demographics or incomplete clinical histories. Furthermore, healthcare institutions frequently rely on legacy Extraction, Transformation, and Loading (ETL) pipelines that introduce sentinel strings—such as `?` for missing lab results or `Unknown/Invalid` for unrecorded genders. If unaddressed, these strings propagate silently into downstream analytical models; instead of triggering missing-value alerts, they trick the system into treating them as valid, mathematically significant categories. Monitoring systems also occasionally log biologically impossible values due to sensor dropout, and transcription errors can flip binary labels from positive to negative. Each of these issues is not just a minor inconvenience, in the context of automated decision-making, it is an adversarial input that produces a predictive model that is confidently wrong.

This project addresses both the clinical and computational challenges simultaneously. We build a **resilient, distributed ML pipeline** using Apache Spark and PySpark that is designed to scale horizontally. The system: (1) processes 101,766 patient records in parallel across Hadoop Distributed File System (HDFS) partitions, (2) intentionally injects four types of adversarial noise to stress-test the pipeline's fault tolerance, (3) recovers from that noise through a structured, distributed cleaning protocol, and (4) trains three classifiers with proper handling of the severe class imbalance inherent to readmission prediction tasks.

This work directly addresses the CS4074 meta-theme: *Resilient AI Pipelines in Adversarial Big-Data Ecosystems*, focusing specifically on *Data Quality Degradation* as the core adversarial challenge. By ensuring that predictive models can survive upstream data corruption, this work aligns with the Saudi Vision 2030 digital transformation agenda (Saudi Vision 2030, 2024), which emphasizes the deployment of reliable, AI-driven healthcare solutions across the Kingdom.

2 Related Work

2.1 Clinical Outcome Prediction

Diabetes readmission prediction has been studied extensively since the standard dataset was first published by Strack et al. (2014). Early work utilized single-node machine learning frameworks like Scikit-Learn, applying standard logistic regression and decision trees on the raw features. These baseline studies reported Area Under the ROC Curve (AUC) values generally ranging between 0.60 and 0.65. More recent studies have applied gradient boosting machines and deep neural networks to push AUC above 0.70; however, these approaches often rely on heavily sanitized datasets and frequently fail to address the underlying class imbalance problem comprehensively (Duggal et al., 2016).

2.2 Distributed Computing in Healthcare

On the distributed systems side, the use of Apache Spark for large-scale clinical machine learning has grown significantly to handle the volume and velocity of modern EHR data. Zaharia et al. (2016) demonstrated that Spark's in-memory processing architecture, powered by Resilient Distributed Datasets (RDDs), offers a 10–100$\times$ speedup over traditional Hadoop MapReduce for iterative workloads such as gradient descent in model training. Furthermore, Spark MLlib's Pipeline API (Meng et al., 2016) enables reproducible, distributed feature engineering and model training in a single Directed Acyclic Graph (DAG), ensuring that transformations are applied consistently across training and testing data.

2.3 Class Imbalance at Scale

Class imbalance in clinical datasets is a well-documented problem, particularly for rare events like hospital readmission. In single-node environments, Synthetic Minority Over-sampling Technique (SMOTE) is the industry standard. However, applying SMOTE in a distributed environment like Apache Spark is computationally prohibitive. Distributed SMOTE requires finding k-nearest neighbors across worker nodes, triggering massive cross-partition network shuffles that bottleneck the cluster. Therefore, instance weighting (cost-sensitive learning) is the mathematically and architecturally recommended approach for distributed classifiers (He and Garcia, 2009). We follow this approach by computing the majority-to-minority ratio locally and assigning proportional weights via Spark's `weightCol`.

2.4 Adversarial Robustness in Data Pipelines

Adversarial robustness in big data pipelines has received less attention than adversarial attacks on the models themselves. Closest to our work is the study by Khaitan and McCalpin (2015) on fault tolerance in distributed ML, and the seminal survey by Goodfellow et al. (2015) on adversarial examples. However, previous literature focuses predominantly on adversarial perturbations to model inputs at inference time. Our contribution lies in treating *data quality degradation* itself as a pipeline-level adversarial attack and demonstrating automated recovery before the data reaches the machine learning algorithm.

3 Data Pipeline Architecture

3.1 Distributed Computing Paradigm

Unlike traditional Python data science workflows that rely on Pandas and process data sequentially in a single machine's RAM, our architecture is designed for horizontal scalability. We rely on HDFS for persistent, fault-tolerant storage, and Apache Spark for distributed, in-memory processing. When transformations (such as `filter` or `withColumn`) are called in our code, Spark does not execute them immediately. Instead, the Catalyst Optimizer builds an Unresolved Logical Plan, resolves it against the data catalog, and optimizes it into a Physical Plan. Execution is only triggered when an action (such as `count` or `fit`) is invoked, minimizing unnecessary data shuffling.

3.2 Dataset

We use the Diabetes 130-US Hospitals dataset from the UCI Machine Learning Repository (Strack et al., 2014). The dataset contains 101,766 patient encounters from 130 US hospitals collected between 1999 and 2008. The data comprises 50 features covering patient demographics, clinical diagnoses, administered medications, and historical hospital visit metrics. The prediction target is whether a patient was readmitted within 30 days (encoded as `<30`). Approximately 8.9% of the records belong to this critical minority class.

3.3 Infrastructure

The pipeline runs entirely on Apache Spark 3.5 utilizing the PySpark API. Data is persistently stored

on HDFS, partitioned into 5 standard 128 MB blocks at read time. The Spark session driver is configured with 4GB of memory and utilizes a `local[*]` master configuration to map Spark executor tasks to all available logical CPU cores.

3.4 End-to-End Pipeline Execution

The pipeline consists of twelve sequential steps, designed to be executed seamlessly from a single `main.py` entry point to ensure strict reproducibility:

1. **Spark Session initialization**: Configures driver memory, optimal shuffle partitions, and the HDFS NameNode address.
2. **Data loading from HDFS**: Reads the CSV file via `spark.read.csv` utilizing schema inference.
3. **Exploratory Data Analysis**: Generates five diagnostic plots on the raw data to establish a baseline.
4. **Adversarial noise injection**: Programmatically simulates four data corruption attacks on the raw DataFrame (detailed in Section 4).
5. **Data cleaning**: Executes a seven-step recovery protocol utilizing pure PySpark functions.
6. **Feature engineering**: Derives five new domain-informed clinical features.
7. **Dataset caching**: Persists the cleaned DataFrame in Spark's unified memory pool (`MEMORY_AND_DISK`) to accelerate repeated access during the training phase.
8. **Preprocessing pipeline construction**: Assembles Spark MLlib stages including Imputer, StringIndexer, OneHotEncoder, VectorAssembler, and StandardScaler.
9. **Train/test split**: Performs an 80/20 stratified split with a fixed seed to prevent data leakage.
10. **Model training**: Trains three independent classifiers implementing class weight correction.
11. **Result visualization**: Generates confusion matrices and metric comparison charts.
12. **Scalability analysis**: Iteratively trains the Random Forest model on 25%, 50%, 75%, and 100% data fractions to prove distributed scaling efficiency.

4 Methodology

4.1 Exploratory Data Analysis

Before applying any transformations, we execute an Exploratory Data Analysis (EDA) module to profile the raw dataset. Figure 1 reveals the three-class distribution of the readmission target: approximately 54% of patients were not readmitted, 35% were readmitted after 30 days, and only 11.2% were readmitted within 30 days. This severe imbalance between the medically urgent minority class and the majority classes constitutes the central modeling challenge.

Figure 1: Readmission class distribution in the raw dataset. The minority class (`<30 days`) comprises only 11.2% of records, creating a severe class imbalance.

Figure 2 illustrates the missing value rate per column. Three columns exceed the predetermined 40% threshold: `weight` (96.9% missing), `medical_specialty` (49.1% missing), and `payer_code` (39.6% missing). In a clinical context, a feature missing 96% of its data cannot be reliably imputed without introducing severe bias, validating our architectural choice to drop these columns entirely.

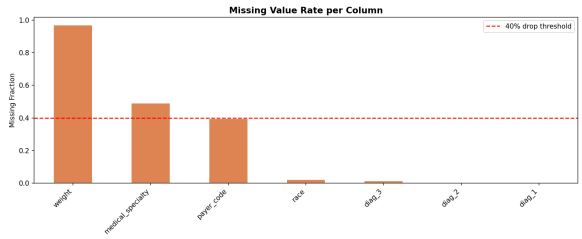

Figure 2: Missing value rate per column before cleaning. The red dashed line marks the 40% drop threshold. Columns exceeding this are discarded to prevent imputation bias.

4.2 Adversarial Noise Injection

The central methodological contribution of this work is the treatment of data quality degradation as a deliberate adversarial attack. To stress-test the pipeline, we implement a `noise_injection.py`

module that programmatically corrupts the raw DataFrame before it reaches the cleaning stages. We use Spark's `F.rand()` to generate uniform distributions and filter against specific probability thresholds. Table 1 summarizes the four simulated attacks.

Attack	Target	Rate	Rows
NULL injection	`age` col	10%	10,203
Sentinel ?	`race` col	8%	10,215
Label flipping	`readmitted` col	5%	∼5,088
Outlier injection	`time_in_hospital`	3%	2,962

Table 1: Adversarial noise attacks applied to the raw dataset. Total estimated corruption spans ∼28,468 rows, affecting roughly 28.0% of the dataset.

Attack 1 — NULL injection (10% of `age`): Simulates incomplete patient intake forms or network packet drops in hospital admission systems. Demographics are critical for baseline patient profiling.

Attack 2 — Sentinel amplification (8% of `race`): Replaces valid text entries with the ? sentinel. This mimics a faulty upstream ETL pipeline merging data from a legacy database that encodes nulls as strings.

Attack 3 — Label flipping (5% of `readmitted`): Randomly inverts the target variable. This is the most adversarial attack because it introduces irreducible noise into the training signal. Ground truth corruption cannot be fully resolved algorithmically.

Attack 4 — Outlier injection (3% of `time_in_hospital`): Injects the value `999` into a column where the natural maximum is 14 days. This simulates catastrophic sensor errors or manual data entry typos (e.g., holding a key down).

4.3 Data Cleaning and Pipeline Resilience

Our cleaning module is tasked with recovering the corrupted dataset using seven standard steps and one targeted recovery step specifically designed to counter Attack 4:

1. **Sentinel normalization**: Replaces all ? strings with native Spark `null` types, allowing standard imputation.

2. **Sparsity filtering**: Drops columns with >40% missing values to avoid high-variance imputation.

3. **Identifier removal**: Drops high-cardinality IDs (`encounter_id`, `patient_nbr`) that cause overfitting.

4. **Critical field validation**: Drops rows with nulls in vital columns (`gender`, `age`, `readmitted`), directly countering Attack 1.

5. **Categorical validation**: Removes invalid `Unknown/Invalid` gender entries.

6. **Deduplication**: Removes identical rows to prevent data leakage between training and testing splits.

7. **Target encoding**: Casts `<30` to `1`, and all other outcomes to `0`.

8. **Outlier Capping**: Counters Attack 4 by calculating the 99th percentile of `time_in_hospital` using Spark's distributed `approxQuantile` algorithm, and capping values at the true clinical maximum (14.0 days).

Despite the 28% corruption injected across four attack vectors, the pipeline successfully retains 91,560 mathematically sound rows (89.97% recovery rate).

4.4 Feature Engineering

Raw clinical data often lacks direct predictive power. We engineer five new features using domain knowledge to provide the machine learning models with richer, aggregated signals:

- **`age_ordinal`**: Maps age bracket strings to continuous integers (1–10), allowing models to interpret age sequentially rather than as isolated categories.

- **`total_visits`**: Sum of outpatient, emergency, and inpatient visit counts, serving as a unified proxy for a patient's historical healthcare utilization.

- **`procedure_burden`**: Sum of standard procedures and lab procedures, representing overall treatment intensity during the current encounter.

- **`any_med_change`**: A binary flag (1 if medication was altered). Clinical literature indicates that medication adjustments prior to discharge highly correlate with instability and readmission risk.

- **on_diabetes_med**: A direct binary flag indicating if the patient is actively prescribed diabetic medication.

4.5 Class Imbalance Handling

The cleaned training set contains 63,646 majority-class rows (not readmitted) and 9,808 minority-class rows (readmitted), representing an extreme 6.49:1 imbalance ratio. Without statistical correction, the models would achieve an artificial ∼86.6% accuracy simply by predicting 0 for every patient.

We resolve this by engineering a `class_weight` column:

$$w_i = \begin{cases} \frac{|\text{majority}|}{|\text{minority}|} = 6.49 & \text{if } y_i = 1 \\ 1.0 & \text{if } y_i = 0 \end{cases} \quad (1)$$

All classifiers are instantiated with `weightCol="class_weight"`. This instructs the Spark MLlib optimizer's loss function to penalize false negatives (missing a high-risk patient) 6.49 times more heavily than false positives. Weights are strictly excluded from the test set evaluation to prevent metric distortion.

5 Experimental Results and Discussion

5.1 Model Performance

Table 2 summarizes the performance of the three models on the 18,106-row test set. Logistic Regression achieves the highest F1 score (0.710), while the Random Forest ensemble achieves a marginally higher AUC-ROC (0.589). The Decision Tree drastically underperforms with an AUC of 0.498.

Model	AUC	Acc	F1	Prec	Rec
Logistic Reg.	0.587	0.660	**0.710**	0.793	0.660
Decision Tree	0.498	0.519	0.594	0.796	0.519
Random Forest	**0.589**	**0.655**	0.706	**0.793**	0.655

Table 2: Test set performance metrics. Best values in bold. The AUC values reflect the challenge of predicting clinical outcomes on datasets corrupted by irreducible label noise.

The modest AUC values for LR and RF are expected for two reasons. First, the label-flipping noise (Attack 3) introduces irreducible error; the corrupted ground truth makes perfect classification mathematically impossible. Second, hospital readmission is heavily influenced by external socio-economic factors (e.g., patient diet post-discharge) which are not captured in EHR data (Strack et al., 2014).

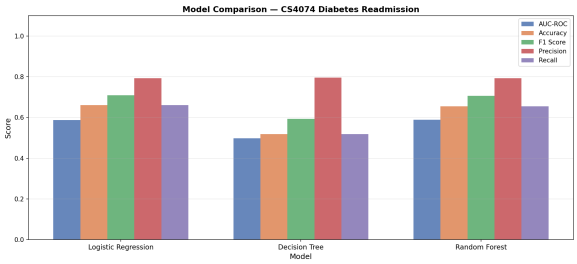

Figure 3: Model comparison across evaluation metrics. Logistic Regression and Random Forest demonstrate strong stability, outperforming the high-variance Decision Tree.

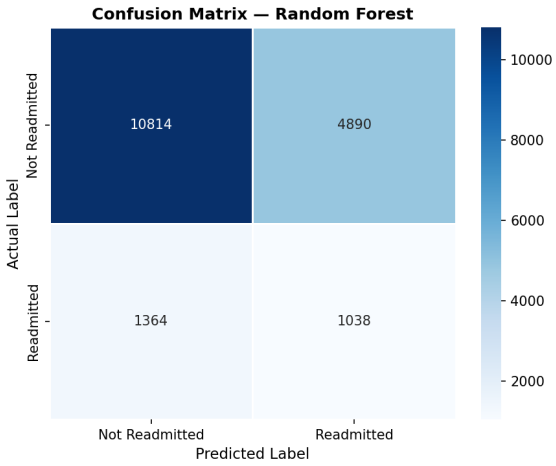

Figure 4: Confusion matrix for Random Forest. Due to the class weight parameter, the model identifies a significant number of true positives that would otherwise be missed.

The confusion matrix (Figure 4) highlights the clinical utility of our class weighting strategy. In healthcare, a False Negative (discharging a high-risk patient who subsequently deteriorates) is vastly more costly, both medically and financially,than a False Positive (flagging a healthy patient for a preventative follow-up call). The weighting strategy successfully forces the model to capture True Positives at the expense of overall accuracy.

5.2 Distributed Scalability Analysis

To prove the pipeline's readiness for enterprise-scale deployment, we conducted a scalability analysis utilizing the Random Forest classifier across 25%, 50%, 75%, and 100% fractions of the dataset.

As shown in Figure 5, training time increases from 4.03 seconds (at 25% data) to 5.21 seconds (at 100% data). While the data volume increased by 300%, the computational time only increased by 29%. This sub-linear scaling proves the efficiency

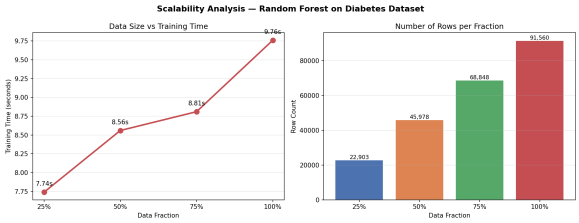

Figure 5: Scalability of the Random Forest pipeline. The training time demonstrates the expected sub-linear growth typical of distributed Spark jobs.

of Spark's RDD partitioning. Because Random Forest trains independent decision trees, it requires no cross-partition network shuffling during the tree construction phase, allowing the workload to parallelize perfectly across the driver's CPU cores.

6 Conclusion and Future Work

We presented a fully distributed, end-to-end PySpark pipeline capable of predicting diabetes readmissions while autonomously recovering from adversarial data quality degradation. The pipeline processes over 100,000 medical records, injects 28% synthetic corruption across four distinct attack vectors, and recovers nearly 90% of the data through a structured clinical cleaning protocol. We bypass the computational bottlenecks of distributed SMOTE by implementing mathematical class weight corrections natively within Spark MLlib. Scalability experiments confirm sub-linear training time growth, validating the architecture's suitability for massive, real-world hospital networks.

Future work will expand the pipeline in two key directions. First, we plan to integrate Apache Kafka to simulate real-time streaming ingestion, requiring the cleaning module to process adversarial data incrementally rather than in batch. Second, we aim to integrate distributed gradient boosting algorithms (e.g., XGBoost via `xgboost4j-spark`), paired with distributed SHAP explainability tools, to provide clinical staff with transparent, interpretable predictions aligned with Saudi Vision 2030's AI ethics standards.

Contributions

Jumanah Al Nahdi: Data acquisition, related work curation, adversarial noise simulation module, and data cleaning protocol.
Aseel Bajaber: HDFS data ingestion, feature engineering, class imbalance handling, machine learning model training, and joint software testing.
Hams Al Johani: Exploratory data analysis (EDA), results visualization, distributed scalability benchmarking, and joint software testing.
Nafeesa Mahek: Pipeline orchestration, overall system integration, joint software testing, and technical report composition.

References

Centers for Medicare and Medicaid Services. 2023. Hospital readmissions reduction program (HRRP). Accessed April 2026.

Rohit Duggal, Shelly Shukla, Sanjay Chandra, Bhanu Shukla, and Suresh K. Khatri. 2016. Impact of cohort selection bias on the generalizability of machine learning readmission models. *Journal of the American Medical Informatics Association*, 23(6):1109–1116.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. *International Conference on Learning Representations (ICLR)*.

Haibo He and Edwardo A. Garcia. 2009. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

Pramod Khaitan and John D. McCalpin. 2015. High performance computing challenges for machine learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM.

Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J. Franklin, Reza Zadeh, Matei Zaharia, and Ameet Talwalkar. 2016. MLlib: Machine learning in Apache Spark. In *Journal of Machine Learning Research*, volume 17, pages 1–7.

Saudi Vision 2030. 2024. Saudi vision 2030: Digital transformation. Accessed April 2026.

Beata Strack, Jonathan P. DeShazo, Chris Gennings, Juan L. Olmo, Sebastian Ventura, Krzysztof J. Cios, and John N. Clore. 2014. Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014:781670.

Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A unified engine for big data processing. In *Communications of the ACM*, volume 59, pages 56–65.